Distributed gradient descent: splitting the batch

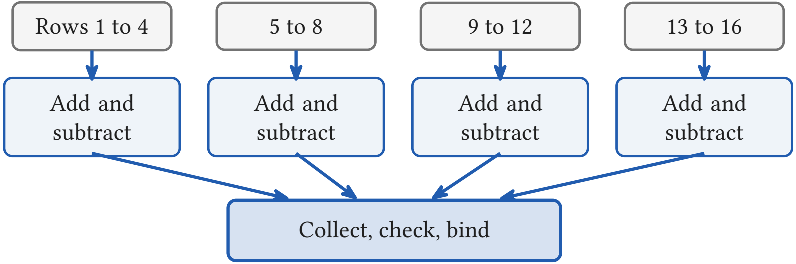
Part 1 of 2: when the model fits and the data does not

Carlos Cotrini

Section 1

The gradient of a mean

Paris, 1793: one task, twenty computers



- The same arithmetic in every hand, a different slice of the table
- Not a pin factory: nobody specialises in a step
- Roegel 2010, from the payrolls: about twenty computers, not the sixty to eighty Prony recalled.

The sum splits, exactly

$$\nabla L(w) \;=\; \underbrace{\frac{1}{P}\sum_{p=1}^{P} g_p}_{\text{the only thing the workers must share}}$$

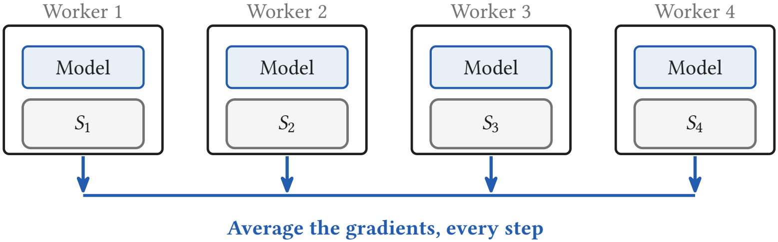
The batch, n examples

S_1	S_2	S_3	S_4

P workers, n/P examples each

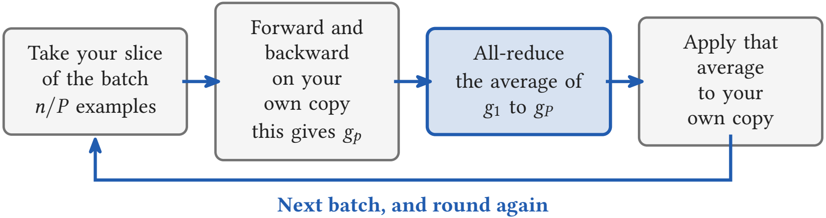
- Equal shards, so every worker holds the same number of examples
- You already have this from Weekend 0. Nothing here is new mathematics.

Every worker holds the whole model



- Data parallelism: replicate the model, split the batch
- The copies stay identical because every worker applies the same average
- This hour assumes the model fits on one device

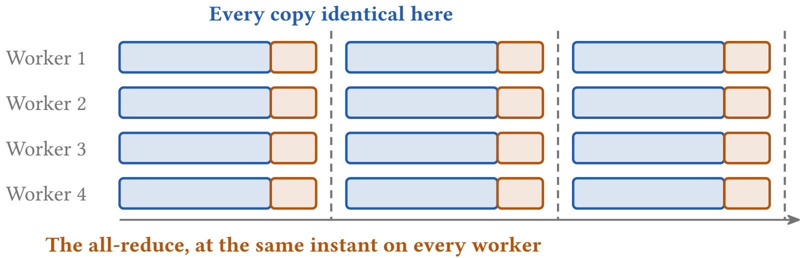
One step of training, run on every worker at once



Three of the four stages are local. The all-reduce is the one that touches the network.

- Every worker runs this same loop, at the same time, on its own slice
- Split the data, aggregate the gradient, and repeat

One all-reduce per step, for every step of the run



- The average lands before the weights move, so every step starts from one model
- Local SGD relaxes this and aggregates every few steps. Everything in this hour is the synchronous version.

Section 2

Four workers, eight examples

2

Four workers, eight examples

Eight examples, one gradient

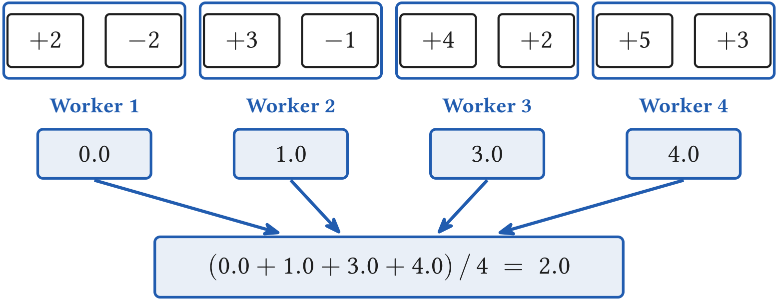
1	2	3	4	5	6	7	8
$+2$	-2	$+3$	-1	$+4$	$+2$	$+5$	$+3$

$$\text{Sum} = 16$$

$$\boxed{16/8 = 2.0}$$

- The single machine answer is 2.0
- Constructed for this lecture. A real gradient is a vector with one number per parameter; one number per example keeps the arithmetic hand checkable.

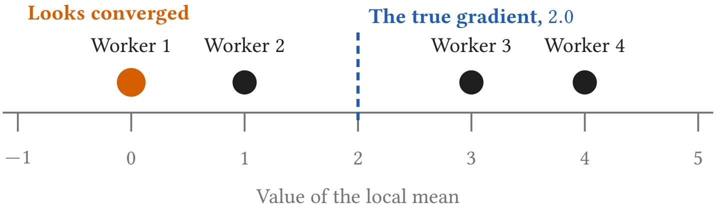
Four workers, two examples each



The same number the single machine got

Exactly equal, not approximately: splitting the batch is a regrouping of the same sum.

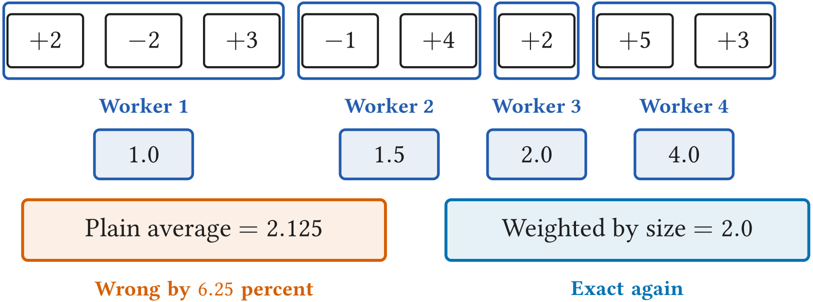
Worker 1 sees zero



- Worker 1's local mean is zero. Its own shard looks converged.
- The global answer is 2.0, and no worker holds it
- That is why the averaging step exists, and why it cannot be skipped

Four workers, eight examples

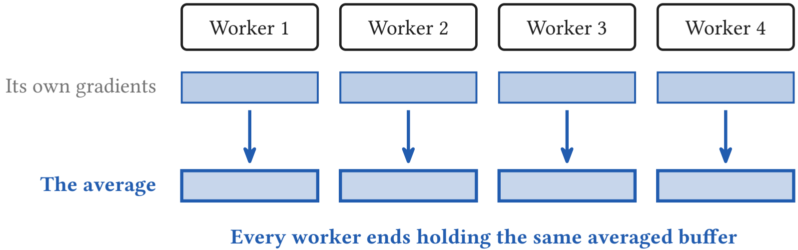
Unequal shards break the identity



Every framework therefore fixes the per worker micro batch and drops or pads the ragged tail.

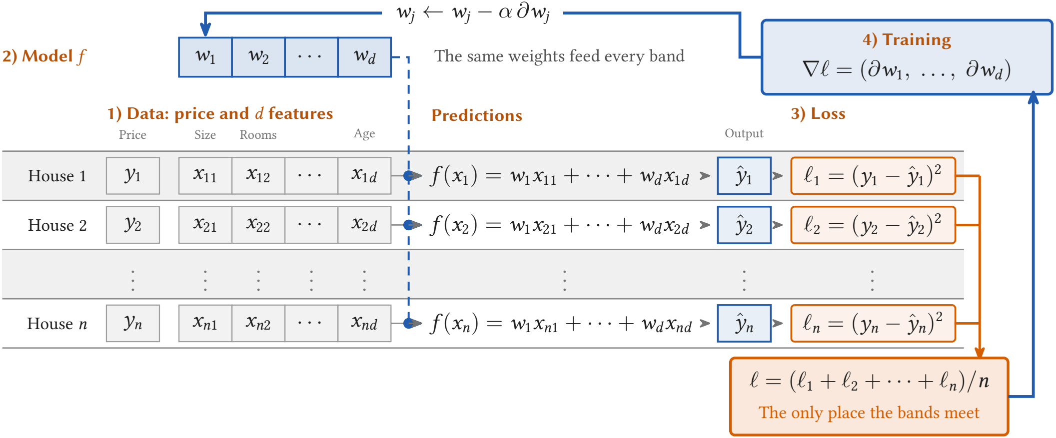
Four workers, eight examples

One buffer, 16.11 gigabytes



- Apertus-8B in bf16: 2 bytes $\times$ 8.053 $\times$ 10^9 parameters $=$ 16.11 GB
- Write K for the size of that buffer, and P for the number of workers
- Compute is local. This is the only part that is not.

One training step of linear regression

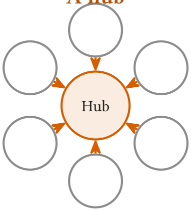


Section 3

The all-reduce

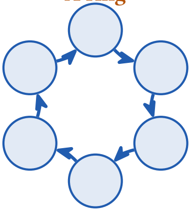
Two ways to average: a hub, or a ring

A hub



$P \times K$ **arrives at one machine**

A ring



$\frac{2(P-1)}{P} \times K$ **leaves each worker**

K is the gradient buffer, 16.11 GB. P is the number of workers.

- The hub's traffic grows with every worker you add

Round the ring, adding

	Worker 1	Worker 2	Worker 3	Worker 4
Chunk 0	a_0	a_0+b_0	$a_0+b_0+c_0$	$a_0+b_0+c_0+d_0$
Chunk 1	$a_1+b_1+c_1+d_1$	b_1	b_1+c_1	$b_1+c_1+d_1$
Chunk 2	$a_2+c_2+d_2$	$a_2+b_2+c_2+d_2$	c_2	c_2+d_2
Chunk 3	a_3+d_3	$a_3+b_3+d_3$	$a_3+b_3+c_3+d_3$	d_3

Round 3: each worker now owns one chunk of the true sum

Chunk j is the same slice of the buffer on every worker, so a_j is worker 1's chunk j, b_j worker 2's, and worker 4 sends back to worker 1.

Round the ring again, copying

	Worker 1 →	Worker 2 →	Worker 3 →	Worker 4
Chunk 0	$a_0+b_0+c_0+d_0$	$a_0+b_0+c_0+d_0$	$a_0+b_0+c_0+d_0$	$a_0+b_0+c_0+d_0$
Chunk 1	$a_1+b_1+c_1+d_1$	$a_1+b_1+c_1+d_1$	$a_1+b_1+c_1+d_1$	$a_1+b_1+c_1+d_1$
Chunk 2	$a_2+b_2+c_2+d_2$	$a_2+b_2+c_2+d_2$	$a_2+b_2+c_2+d_2$	$a_2+b_2+c_2+d_2$
Chunk 3	$a_3+b_3+c_3+d_3$	$a_3+b_3+c_3+d_3$	$a_3+b_3+c_3+d_3$	$a_3+b_3+c_3+d_3$

Round 6: every worker now holds the whole averaged buffer

Every worker holds the whole model, so the copies stay identical only if every worker applies the same whole average

Same ring, same traffic, one change: the arriving chunk replaces instead of adding. The four rows are four different slices, so a column of four cells is the whole averaged buffer.

Two times P minus one rounds

$$\text{bytes per worker} = \underbrace{\frac{2(P-1)}{P}}_{\text{always below 2}} \times \underbrace{K}_{\text{the whole buffer}}$$

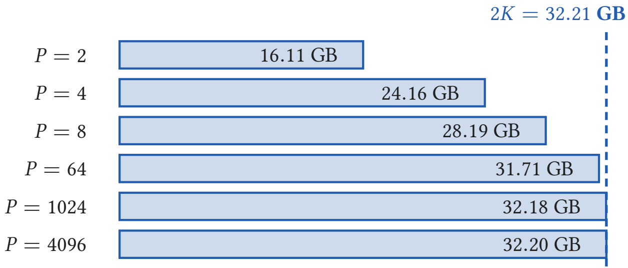
$P-1$ rounds adding, then $P-1$ copying

A chunk has to reach the other $P-1$ workers, so one pass round the ring is $P-1$ rounds, three of them at $P=4$

- The factor never reaches 2, so the traffic never reaches $2K$
- Patarasuk and Yuan 2009 proved this is the least any all-reduce can move.

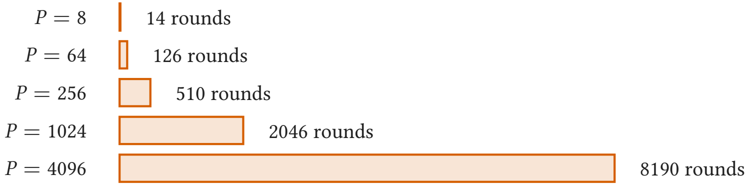
The all-reduce

The bytes barely grow



- 512 times the workers, 14.3 percent more bytes each
- Apertus-8B gradients in bf16, so $K = 16.11$ GB.

The rounds grow linearly in P



Each round pays a latency, whatever it carries

- The bytes barely grow. The rounds grow linearly.
- Past 512 GPUs the latency no longer hides behind the backward pass
- The ring is not Baidu's invention: it is Patarasuk and Yuan 2009.

Section 4

Where it stops scaling

One step, two terms

$T(P)$ is the wall clock time for *one training step* on P workers

$$T(P) = \underbrace{\frac{T(1)}{P}}_{\text{falls with } P} + \underbrace{\frac{2(P-1)}{P} \cdot \frac{K}{B}}_{\text{does not fall at all}}$$

Work per step: 8 million tokens $\times$ $6N = 3.35 \times 10^{17}$ FLOP

Per GPU rate: 990 TFLOP/s dense BF16 per GPU, at an MFU of 0.30

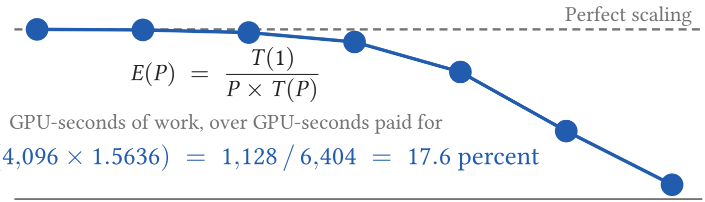
So $T(1)$ is: 1128 seconds, about 19 minutes, on one GPU

The buffer K: 16.11 GB, the gradients of Apertus-8B in bf16

The link B: 25 GB/s per GPU, Slingshot-11 on Alps

$$T(1024) = \underbrace{1128/1024}_{\text{compute}} + \underbrace{1.29}_{\text{all-reduce}} = 2.39 \text{ seconds}$$

93 percent at 64 workers, 18 at 4,096



$$E(P) = \frac{T(1)}{P \times T(P)}$$

GPU-seconds of work, over GPU-seconds paid for

$$1{,}128 \,/\, (4{,}096 \times 1.5636) = 1{,}128 \,/\, 6{,}404 = 17.6 \text{ percent}$$

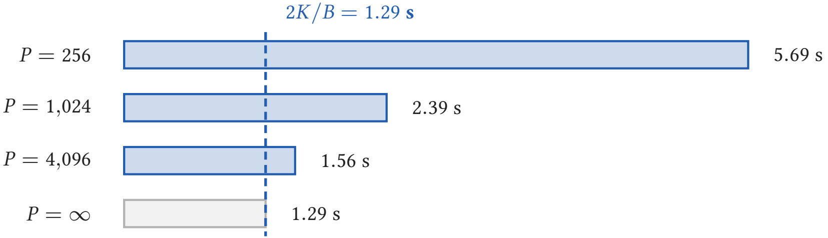
Workers	1	4	16	64	256	1,024	4,096
Efficiency, percent	**100**	**99.7**	**98.3**	**93.3**	**77.4**	**46.1**	**17.6**

- So 82 percent of the cluster-seconds in that step were spent waiting
- At 1,024 workers, communication is 54 percent of the step
- The first term falls as one over P. The second does not fall at all.

- These are the model's numbers, charging the all-reduce after the whole backward pass.

Where it stops scaling

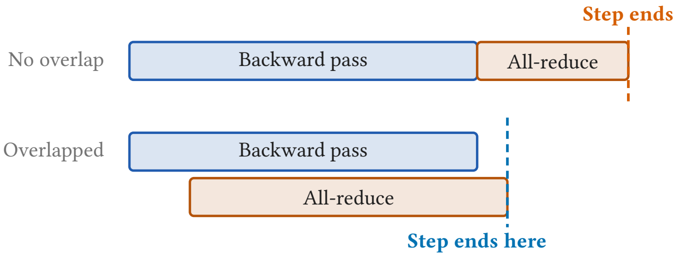
In this model the step never falls below 1.29 seconds



- Compute over P goes to zero, and $2(P-1)/P \times K/B$ goes to $2K/B$
- Four times the machine, 1,024 to 4,096, buys 0.83 s of a 2.39 s step
- Every worker after that, all of them together, buys 0.28 s more

Where it stops scaling

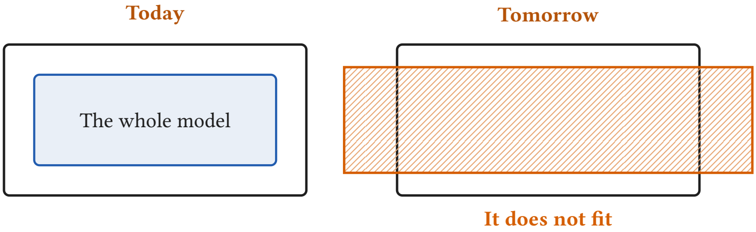
18 percent modelled, 80 percent measured



- The model above charges the all-reduce after the whole backward pass
- Real systems start averaging before the backward pass has finished
- Apertus measured about 80 percent at 4,096 GPUs, not 17.6, so overlapping is worth a factor of about 4.5

Where it stops scaling

Tomorrow the model does not fit



- Everything in this hour assumed one device holds the whole model
- Apertus-70B does not
- Tomorrow at 08:00: cut the model, not the batch